

First Year Master of Technology

Lab Assignment 3**Course Code:** 2304514L**Course Name:** Machine Learning Laboratory**Academic Year:** 2025–26

Student Details

- **Student Name:** Utkarsh Bele
- **PRN Number:** 202503040003

Title: Ensemble Learning and Performance Evaluation on Customer Churn Dataset**Objective**

To develop and analyze multiple regression models for predicting medical insurance costs using a real-world dataset. The objective includes implementing models such as Linear Regression, Decision Tree, and Random Forest, evaluating their performance using appropriate metrics (MAE, MSE, RMSE, and R^2), and comparing their effectiveness to identify the most suitable model for accurate prediction.

Problem Statement

1. Develop regression models for a real-world application and evaluate their performance using appropriate metrics.
2. Implement Gradient Descent for optimizing Linear Regression and compare it with Analytical Solutions.

Code

```
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
# Base Models
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

```
# Ensemble Models
```

```
from sklearn.ensemble import AdaBoostClassifier, VotingClassifier, StackingClassifier
```

```

# ----- LOAD DATA -----
df = pd.read_csv('/content/WA_Fn-UseC_-Telco-Customer-Churn.csv')

# ----- PREPROCESSING -----

# Convert TotalCharges to numeric
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df.fillna(df.mean(numeric_only=True), inplace=True)

# Encode categorical
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

# Features & Target
X = df.drop('Churn', axis=1)
y = df['Churn']

# Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----- BASE MODELS -----
lr = LogisticRegression(max_iter=1000)
dt = DecisionTreeClassifier(max_depth=5)
svm = SVC(probability=True)

# Train Base Models
lr.fit(X_train, y_train)
dt.fit(X_train, y_train)
svm.fit(X_train, y_train)

# Predictions
lr_pred = lr.predict(X_test)
dt_pred = dt.predict(X_test)
svm_pred = svm.predict(X_test)

# ----- ENSEMBLES -----

# Voting
voting = VotingClassifier(
    estimators=[('lr', lr), ('dt', dt), ('svm', svm)],
    voting='soft'
)

```

```

)
voting.fit(X_train, y_train)
voting_pred = voting.predict(X_test)

# AdaBoost
ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50
)
ada.fit(X_train, y_train)
ada_pred = ada.predict(X_test)

# Stacking
stack = StackingClassifier(
    estimators=[('lr', lr), ('dt', dt), ('svm', svm)],
    final_estimator=LogisticRegression()
)
stack.fit(X_train, y_train)
stack_pred = stack.predict(X_test)

# ----- EVALUATION -----
def evaluate(y_test, y_pred, name):
    print(f"\n{name}")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("Recall:", recall_score(y_test, y_pred))
    print("F1 Score:", f1_score(y_test, y_pred))

# Base Models
evaluate(y_test, lr_pred, "Logistic Regression")
evaluate(y_test, dt_pred, "Decision Tree")
evaluate(y_test, svm_pred, "SVM")

# Ensemble Models
evaluate(y_test, voting_pred, "Voting Classifier")
evaluate(y_test, ada_pred, "AdaBoost")
evaluate(y_test, stack_pred, "Stacking")

```

Output

```
Logistic Regression
Accuracy: 0.8161816891412349
Precision: 0.678125
Recall: 0.5817694369973191
F1 Score: 0.6262626262626263

Decision Tree
Accuracy: 0.794180269694819
Precision: 0.6055979643765903
Recall: 0.6380697050938338
F1 Score: 0.6214099216710183

SVM
Accuracy: 0.8069552874378992
Precision: 0.684981684981685
Recall: 0.5013404825737265
F1 Score: 0.5789473684210527

Voting Classifier
Accuracy: 0.8161816891412349
Precision: 0.7111111111111111
Recall: 0.514745308310992
F1 Score: 0.5972006220839814

AdaBoost
Accuracy: 0.8062455642299503
Precision: 0.6623376623376623
Recall: 0.546916890080429
F1 Score: 0.5991189427312775

Stacking
Accuracy: 0.8126330731014905
Precision: 0.6626865671641791
Recall: 0.5951742627345844
F1 Score: 0.6271186440677966
```

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.816	0.678	0.582	0.626
Decision Tree	0.794	0.606	0.638	0.621
SVM	0.807	0.685	0.501	0.579
Voting	0.816	0.711	0.515	0.597
AdaBoost	0.806	0.662	0.547	0.599
Stacking	0.813	0.663	0.595	0.627

1. Overall Best Model

Stacking performed best overall

- Highest **F1 Score (0.627)**
- Balanced Precision + Recall

Meaning:

Stacking provides the best trade-off between detecting churn customers and avoiding false alarms.

2. Logistic Regression (Baseline)

- Highest **Accuracy (0.816)** (tie with Voting)
- Decent F1 Score

Insight:

A simple model performing competitively indicates the dataset has **linear patterns**.

3. Decision Tree

- Highest **Recall (0.638)**

Meaning:

Best at identifying churn customers (fewer false negatives)

But:

- Lower precision → more false positives

4. SVM

- Good precision but **very low recall (0.50)**

Meaning:

Conservative model — predicts churn only when very sure

5. Voting Classifier

- Highest **Precision (0.711)**

Meaning:

Makes fewer false positive predictions

But:

- Lower recall → misses churn cases

6. AdaBoost

- Balanced but not best in any metric

Insight:

Improves weak learners but limited due to dataset complexity

7. Stacking

- Best **F1 Score**
- Good balance of precision and recall

Key Insight:

Learns how to combine models optimally → better than simple averaging

“Ensemble methods do not always guarantee significant improvement; their effectiveness depends on diversity and strength of base models.”

Writes on following points

1. What is the difference between simple linear regression and multiple linear regression?

Simple Linear Regression (SLR):

Uses **one independent variable** to predict a dependent variable.

Equation:

$$y = \beta_0 + \beta_1 x$$

Multiple Linear Regression (MLR):

Uses **two or more independent variables** to predict the dependent variable.

Equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Key Difference:

SLR models a **single-variable relationship**, while MLR captures **multi-variable interactions**, making it more realistic for complex datasets.

2. Why do we use performance metrics like MSE, RMSE, MAE, and R² to evaluate regression models?

These metrics quantify how well a regression model fits the data:

- **MSE (Mean Squared Error):**
Measures average squared difference between actual and predicted values. Penalizes large errors more heavily.
- **RMSE (Root Mean Squared Error):**
Square root of MSE; gives error in the same units as the target variable, making it interpretable.
- **MAE (Mean Absolute Error):**
Measures average absolute difference. Less sensitive to outliers compared to MSE.
- **R² (Coefficient of Determination):**
Indicates how much variance in the dependent variable is explained by the model (ranges from 0 to 1).
- **Purpose:**
These metrics help compare models, diagnose errors, and ensure the model generalizes well.

3. How does Gradient Descent optimize the parameters of a regression model?

Gradient Descent is an iterative optimization algorithm used to minimize the cost function.

- It starts with **initial parameter values** (weights).
- Computes the **gradient (slope)** of the cost function.
- Updates parameters in the **opposite direction of the gradient** to reduce error.

$$\theta = \theta - \alpha \cdot \nabla J(\theta)$$

Where:

- θ = parameters (weights)
- α = learning rate
- $\nabla J(\theta)$ = gradient of cost function
- This process repeats until the model reaches the **minimum error (convergence)**.

4. Can you explain the difference between Gradient Descent and the Analytical (Closed-form) solution for Linear Regression?

Gradient Descent:

- Iterative method
- Works well for **large datasets**

- Requires tuning (learning rate, iterations)
- Approximate solution

Analytical Solution (Normal Equation):

- Direct mathematical computation (no iteration)

$$\theta = (X^T X)^{-1} X^T y$$

Gives **exact optimal solution**.

- Computationally expensive for large datasets due to matrix inversion
- **Key Difference:**
Gradient Descent is **scalable and flexible**, while the analytical method is **exact but not efficient for large-scale data**.

5. What factors influence the convergence of Gradient Descent?

Several factors determine how fast and effectively Gradient Descent converges:

- **Learning Rate (α):**
 - Too high → may overshoot and diverge.
 - Too low → very slow convergence
- **Feature Scaling:**
 - Unscaled features cause uneven convergence.
 - Normalization/standardization improves speed.
- **Initial Values:**
 - Poor initialization can slow convergence or lead to local minima.
- **Cost Function Shape:**
 - Convex functions → guaranteed global minimum
 - Non-convex → risk of local minima
- **Data Quality:**
 - Noise and outliers can affect convergence.
- **Number of Iterations:**
 - Too few → underfitting

Too many → unnecessary computation

Conclusion:

The practical demonstrated the implementation of ensemble learning techniques on the Customer Churn dataset. Logistic Regression performed competitively, indicating underlying linear patterns in the data. Among ensemble methods, Stacking achieved the best overall performance with the highest F1 score, providing a balanced trade-off between precision and recall. Voting classifier improved precision but reduced recall, while AdaBoost showed moderate improvements.

The results highlight that ensemble methods do not always significantly outperform base models, and their effectiveness depends on the dataset and diversity of base learners. Stacking proved to be the most effective technique due to its ability to learn optimal combinations of multiple models.